MCPulse --- Updated Master Implementation Prompt

Project Overview

MCPulse is a Cloud Computing semester project that monitors remotely accessible Model Context Protocol (MCP) servers through supported remote transports, especially Streamable HTTP and Server-Sent Events (SSE), as supported by each endpoint.

The project currently provides server-level monitoring. The requested extension is to add tool-level monitoring and analysis for exactly these five services:

1. GitHub
2. Google Drive
3. Brave Search
4. Cloudflare
5. Hugging Face

Do not add any other service integrations. Do not substitute or expand this list without explicit approval.

The project must use provider-hosted remote MCP endpoints where available. We do not want to run the underlying applications or their MCP servers locally or host those applications ourselves. Verify endpoint availability, transport compatibility, authentication requirements, permitted tools, quotas, and costs before implementing each integration. If a provider does not offer a suitable remote MCP endpoint accessible to this project, report the limitation rather than pretending it is supported or silently building a locally hosted alternative.

Existing Project --- Preserve It

Current stack

- Frontend: `apps/web` --- React 18, TypeScript, Vite, Tailwind CSS, Recharts
- Backend: `apps/api` --- Node.js, Express 5, TypeScript, `node-cron`
- Database: PostgreSQL 16 through Docker
- Frontend development port: `5173`
- Backend development port: `3001`

Existing database tables

- `mcp_servers`: registered servers and configuration
- `monitoring_checks`: server-level check history, duration, outcome, status/error fields
- `incidents`: server-level incidents and resolution history

Inspect the real schema and migrations before making changes.

Existing backend functionality

- MCP JSON-RPC `initialize` handshake in `apps/api/src/monitoring/checker.ts`
- Existing handling of supported Streamable HTTP/SSE responses
- Latency measurement and sanitized errors
- Scheduler in `apps/api/src/monitoring/scheduler.ts`, polling every 10 seconds and checking servers according to their configured intervals
- Prevention of overlapping checks for the same server
- Server incident opens after three consecutive failures and resolves after a successful check
- Existing REST endpoints:
  - `GET /health`
  - `GET /api/dashboard/summary`

- `POST /api/servers`
- `DELETE /api/servers/:id`
- `POST /api/servers/:id/check-now`
- `GET /api/servers/:id/checks`

Existing frontend functionality

- Dashboard summary cards
- Server table, status indicators, last check, and response time
- Add Server modal with validation and current presets
- Manual Check Now and Delete actions
- Auto-refresh controls
- Response-time chart and incidents list

Known issues

1. The chart currently uses the first server in the list rather than the selected server.
2. Unexpected API errors need better development diagnostics without exposing internal details in production.
3. AWS deployment has not yet been implemented.

Preserve existing functionality and compatibility. Do not rebuild the project from scratch.

Goal of the Extension

Extend MCPulse from server-level monitoring to **MCP tool-level monitoring** for the five specified services. The platform must measure the remote MCP endpoint and approved tool calls, record metrics over time, validate responses against explicit rules, and present historical results.

The system must distinguish:

- MCP endpoint availability and initialization
- Tool discovery status
- Individual tool-call success or failure
- Response latency
- Response-validation outcomes
- AI-generated interpretations of monitoring evidence

A reachable MCP endpoint does not prove that every tool or the underlying application is healthy.

The Five Required Services

Implement support for these five service categories only. Use provider-hosted remote MCP endpoints when available; do not host the applications or their MCP servers ourselves.

1. GitHub

Intended tool examples, subject to the actual remote endpoint and permissions:

- Search repositories
- Retrieve issues
- Read pull-request details

Monitor:

- Endpoint availability and initialization latency
- Tool-call duration and success rate
- Authentication, rate-limit, and timeout errors

- Whether expected repository/issue fields are returned

Prefer read-only operations.

2. Google Drive

Intended tool examples, subject to actual endpoint access and permissions:

- Search files
- Retrieve file metadata
- Read an approved test document

Monitor:

- Tool-call latency and availability
- OAuth and permission errors
- Expected metadata fields
- Empty results versus actual execution failures

Verify remote endpoint availability, preview eligibility, authentication, and quotas before implementation.

3. Brave Search

Intended tool examples, subject to actual endpoint access and permissions:

- Search a known topic
- Retrieve result titles and URLs
- Validate results against predefined query criteria

Monitor:

- Search latency and errors
- Number of results returned
- Required response fields and URL validity
- Basic relevance against predefined test queries

Verify API access, quota, pricing, remote MCP endpoint availability, and transport compatibility.

4. Cloudflare

Intended tool examples, subject to actual endpoint access and permissions:

- Retrieve permitted account or zone information
- Inspect DNS or configuration metadata where supported
- Query available infrastructure information

Monitor:

- Availability and tool-call latency
- Authentication and permission errors
- Response completeness
- Changes in results across repeated checks

Use read-only permissions wherever possible. Available tools depend on the selected Cloudflare remote MCP endpoint and account permissions.

5. Hugging Face

Intended tool examples, subject to actual endpoint access and permissions:

- Search models

- Search datasets
- Retrieve model or dataset metadata

Monitor:

- Search and metadata-retrieval latency
- Tool success and timeout rates
- Presence of expected metadata fields
- Whether returned resources match basic query criteria

Prefer Hub search and metadata tools initially. Do not add inference, hosted jobs, or other extra functionality unless explicitly approved. Verify remote endpoint access, credentials, quotas, and potential costs.

Shared Monitoring Capabilities

Use one reusable MCP client and monitoring engine across all five services. Do not build five unrelated monitoring systems.

For each registered remote MCP server:

1. Connect using a supported remote transport.
2. Perform the correct MCP initialization and session handling.
3. Discover tools using `tools/list`, including pagination where required.
4. Store tool metadata and input schemas where available.
5. Allow users to select tools and configure explicit, approved test profiles.
6. Execute only configured tool calls through `tools/call`.
7. Measure end-to-end call duration.
8. Record outcome, sanitized error category/details, and response-validation outcome.
9. Store history for charts, metrics, and incidents.
10. Support manual testing and safe scheduled checks.

Do not automatically execute every discovered tool. Do not invent inputs and send them to real external applications without explicit configuration. Begin with read-only tools. Any potentially side-effecting action must remain disabled unless explicitly authorized and safely controlled.

Metrics and Reporting

Expose the following where sufficient data exists:

- Server initialization latency and availability
- Tool-call latency
- Success/failure rate
- Median latency and p95 latency
- Timeout count and error categories
- Response-validation outcomes
- Last successful and failed checks
- Historical trends
- Server-level and tool-level incidents

Explain the calculation window and sample count for aggregated metrics. Handle empty histories and insufficient samples without fabricating values.

Keep server health and tool health separate. A server can be reachable while a specific tool is failing.

AI Analysis

Add AI analysis only after deterministic tool monitoring and history work reliably.

The AI service may analyze bounded, sanitized evidence such as:

- Relevant server and tool check records
- Latency samples and trends
- Error categories and validation failures
- Incident history and tool metadata

AI output may include:

- A concise summary
- Observed facts supported by the collected data
- Possible causes, explicitly labeled as hypotheses
- Uncertainty notes
- Suggested troubleshooting steps

Requirements:

- Never present a suspected cause as confirmed without evidence.
- Clearly distinguish measured facts from model-generated interpretation.
- Link reports to the checks or incidents analyzed.
- Never send tokens, authorization headers, secrets, or unnecessary personal/customer data to the AI provider.
- Do not let the AI invoke MCP tools. Tool execution must remain within the controlled monitoring runner.
- Do not invoke an LLM every 10 seconds. Use on-demand analysis or rate-limited incident/anomaly triggers.
- Keep the AI provider optional and configurable through environment variables.
- The monitoring platform must still work if no AI API key is configured.
- Never commit real credentials.

Database Changes

Inspect the actual schema and migrations first. Add migrations that preserve existing data.

Consider adding these tables, adapted to existing conventions:

`mcp_tools`

- ID and server foreign key
- Tool name and description
- Input schema where available
- Active/removed status
- First-seen and last-seen timestamps
- Metadata/schema fingerprint
- Created/updated timestamps

`tool_test_profiles`

- ID and tool/server association
- Profile name
- Explicit test input JSON
- Expected-result or validation rules
- Enabled state
- Manual or scheduled execution mode
- Optional interval and safe execution limits
- Created/updated timestamps

`tool_monitoring_checks`

- ID and tool/profile association
- Check timestamp
- Duration in milliseconds
- Outcome and error category/code
- Sanitized error details
- Validation result
- Bounded/redacted response metadata where needed
- Correlation/request ID where useful

`tool_incidents`

Store tool-level incidents without breaking existing server incidents. Prevent duplicate open incidents for the same server/tool condition.

`ai_analysis_reports`

- ID and associated server/tool/incident where applicable
- Analysis type and provider/model identifier
- Sanitized evidence references
- Structured report output
- Created timestamp and status/error metadata

Do not store credentials or unnecessary full tool outputs. Adapt names and fields to the existing schema after inspection.

API Changes

Follow current route, validation, and response conventions. Potential routes include:

- `POST /api/servers/:id/discover-tools`
- `GET /api/servers/:id/tools`
- `GET /api/tools/:id`
- `POST /api/tools/:id/test`
- `GET /api/tools/:id/checks`
- `GET /api/tools/:id/metrics`
- CRUD routes for test profiles
- Routes to request and retrieve AI analysis
- Tool incident listing and detail routes

Preserve existing endpoint contracts unless a documented change is necessary. Validate inputs and IDs. Return structured, safe errors. Keep stack traces and sensitive diagnostics out of production responses.

Frontend Changes

Preserve the existing React/Tailwind design and extend it with:

Tool Explorer

- Select a registered remote MCP server
- Discover or refresh tools
- Search and filter discovered tools
- View descriptions and input schemas
- Display whether a tool is discovered, configured, healthy, failing, or not yet tested

Test Profiles

- Configure explicit, safe test inputs
- Configure validation rules
- Support a manual Test Now action
- Display validation errors, loading states, and final results
- Require explicit confirmation for any approved side-effecting operation

Tool Analytics

- Filter by service/server, tool, and time range
- Plot latency history for the selected tool
- Show success rate, median/p95 latency, failures, timeouts, and validation results
- Handle empty, loading, and error states
- Fix the existing chart-selection bug; do not always use the first server

AI Insights

- Separate observed facts from hypotheses
- Display recommendations and uncertainty
- Link insights to supporting monitoring records
- Show when AI analysis is unavailable or stale

Incidents

- Retain existing server incidents
- Add tool-level incidents
- Link incidents to related checks
- Clearly distinguish endpoint failure from individual tool failure

Do not display fake metrics as real results. Label any demonstration data explicitly.

Scheduler and Reliability

- Preserve the current server monitoring scheduler.
- Add tool monitoring without blocking the Express event loop or starving server checks.
- Prevent overlapping executions for the same test profile.
- Enforce global and per-server concurrency limits.
- Respect provider rate limits and `Retry-After` where applicable.
- Apply timeouts and size limits.
- Retry only when safe; never automatically retry non-idempotent or side-effecting calls.
- Avoid duplicate jobs or incidents after restarts.
- Use configured intervals and safe minimums; do not call every external tool every 10 seconds.
- Support graceful shutdown and structured logging.

Security Requirements

- Because MCPulse accepts remote endpoints, implement SSRF protections.
- Validate endpoint schemes and destinations; block loopback, private, link-local, cloud metadata, and other prohibited destinations unless a narrowly controlled development exception is explicitly required.
- Revalidate redirects or disable redirects where safer.
- Consider DNS rebinding and resolved-address checks.
- Use connection, read, and total request timeouts.

- Limit request, response, and tool-output sizes.
- Enforce rate, concurrency, and execution limits.
- Redact credentials and authorization headers from logs and errors.
- Use explicit tool allowlisting and test profiles.
- Default to read-only operations.
- Never expose a public endpoint that can scan arbitrary hosts.
- Store secrets only through secure configuration; never commit real keys.
- Restrict deployed administrative/test endpoints.

AWS Deployment Constraint

MCPulse is intended to be deployed on AWS, subject to AWS Learner Lab permissions.

Do not deploy the five underlying applications or their MCP servers ourselves. The goal is to connect MCPulse to suitable provider-hosted remote MCP endpoints.

Before implementing each service, verify:

- A provider-hosted remote MCP endpoint is available and accessible.
- It supports a transport and protocol flow compatible with MCPulse.
- Authentication can be configured securely.
- The desired tools are actually exposed.
- Free-tier eligibility, quotas, and potential costs are understood.
- The integration can be tested without hosting the application or MCP server ourselves.

If an integration requires us to host its MCP server, mark it as unsuitable for the current requirement and report the limitation. Do not silently build a local or self-hosted substitute. Do not claim an integration is verified until it has passed an end-to-end test.

Keep the existing Docker workflow for developing MCPulse itself. Use AWS services only where permitted by the Learner Lab, and avoid unnecessary cost.

Required Implementation Workflow

Phase 1 --- Audit only

Inspect the existing repository, package manifests, migrations/schema, checker, scheduler, API routes, frontend, environment examples, and tests.

Identify: - What works - The chart-selection bug - Error-handling gaps - Current transport and authentication capabilities - Security risks - Which existing features must be preserved

Run existing tests, lint, type-check, and build commands where available. **Do not edit files during this initial audit.**

Phase 2 --- Plan

Provide a file-by-file change list, proposed migrations, API contracts, security controls, integration verification findings, dependencies, and a test plan. Do not make substantial edits until the plan is presented.

Phase 3 --- Implement incrementally

1. Fix chart selection and safe development diagnostics.
2. Add database migrations and MCP tool discovery.
3. Add the Tool Explorer UI.
4. Add controlled test profiles and manual tool testing.
5. Add tool metrics, scheduling, and incidents.
6. Add optional AI analysis.

7. Verify GitHub and Hugging Face as initial integration candidates, provided their remote endpoints meet the requirements.
8. Integrate Google Drive, Brave Search, and Cloudflare through the same framework after verifying remote access and costs.
9. Prepare AWS deployment only after local development tests and remote integration tests pass.

After each phase, run relevant tests, type-check, lint, and build. Report actual results; never claim a check passed if it was not run.

Phase 4 --- Documentation

Update the documentation with:

- Architecture and data flow
- Setup and environment variables
- Database migrations
- How to register a remote MCP endpoint
- How to discover tools
- How to configure and run safe tests
- Metric and incident calculations
- AI analysis behavior and limitations
- Exact integration status for each of the five selected services
- Endpoint, transport, authentication, quota, and cost prerequisites
- Security model and known limitations
- Test results
- AWS deployment instructions appropriate to the available Learner Lab permissions

Acceptance Criteria

The implementation is complete only when:

1. Existing server registration, checks, dashboard, history, and incidents still work.
2. The chart shows the selected server/tool's history rather than always the first server.
3. MCPulse can discover and display tools from a compatible remote MCP endpoint.
4. Tool inventory changes are detected and persisted.
5. A user can configure an approved test profile and manually execute a safe tool call.
6. Each test records duration, outcome, sanitized errors, and validation results.
7. Server health and tool health are represented separately.
8. Tool history, metrics, and incidents are available through the API and UI.
9. Optional AI analysis can summarize sanitized evidence, while MCPulse continues to work without an AI key.
10. Only these five integrations are pursued: GitHub, Google Drive, Brave Search, Cloudflare, and Hugging Face.
11. At least two of the five integrations pass end-to-end tests against provider-hosted remote MCP endpoints. Any unverified or unsuitable integration is explicitly marked as such.
12. No underlying application or MCP server is hosted by us as a workaround for missing remote access.
13. Endpoint protections, execution limits, and secret redaction are tested.
14. Existing tests, type-check, lint, and build results are reported accurately.
15. Local MCPulse development and its Docker database workflow remain usable.

Final Instruction

Start with **Phase 1 --- Audit only**. Inspect the existing MCPulse repository and report findings before editing any files. Then propose a minimal-change implementation plan.

Do not add any service integrations beyond the five specified. Do not assume endpoints, free access, authentication, tools, or transports are available until verified. Do not replace the existing project with a new scaffold.

The goal is a secure, maintainable Cloud Computing project that extends existing remote MCP server monitoring with tool-level observability and evidence-based AI analysis, while connecting only to provider-hosted remote MCP endpoints.